

E-Ink Info Dashboard

Chain K — Hardware & Systems Foundations · Full written lesson

E-ink is the opposite of every screen most projects reach for: it draws essentially no power once an image is set, is readable in direct sunlight, and stays legible with the Pi powered off entirely.

Lesson 1 — Bistable E-Ink: Why It Holds an Image With Zero Power

E-ink is the opposite of every screen most projects reach for: once an image is set, it stays there — with the Pi powered off entirely — because of how the display physically works, not because of any clever software trick.

- Each pixel is a microcapsule containing charged black and white particles suspended in fluid. Applying an electric field moves the particles of one color to the top surface, making that pixel appear black or white (or a grey level, on panels that support it).
- This is **bistable**: once the particles have moved into position, they stay there with no power at all. Power is only needed to change the image, never to hold it — the exact opposite of an LCD or OLED, which needs continuous power just to keep displaying anything.
- E-ink is also **reflective**, not backlit — it uses ambient light the way real paper does, which is why it stays legible in direct sunlight and doesn't strain eyes the way a backlit screen can.

The trap: buying an "e-paper" listing with no controller chip specified and no driver library linked. E-ink panels vary enough between controllers that a generic, undocumented one can mean days of driver debugging for a beginner.

Worked example: unplug the Pi entirely once an image is drawn — the panel keeps showing it, unchanged, indefinitely. The exact same test on any LCD or laptop screen goes black the instant power is cut.

Lesson 2 — Partial vs Full Refresh: The Real Ghosting Tradeoff

Two different ways to update the panel exist for a reason — and understanding the actual tradeoff between them is most of what makes an e-ink project usable instead of frustrating.

- A **full refresh** clears the whole panel (often visibly flashing) and redraws it precisely — every pixel lands exactly where it should, with zero residual. It's also slow: several seconds, not milliseconds.
- A **partial refresh** updates just a region, much faster — but the drive waveform doesn't fully migrate every particle in one pass. Each partial refresh only moves a pixel part of the way toward its target, not all the way.
- Against a target that stops changing, repeated partial refreshes do eventually converge — the leftover error shrinks by a fixed fraction each time. But against a target that keeps changing (a clock digit updating every minute), the error never fully clears — that's **ghosting**.

The trap: ghosting from skipping periodic full refreshes in favor of partial-only updates. The fix isn't "a few more partial refreshes" on a target that keeps moving — it's a periodic full refresh, which clears everything in one exact pass regardless of how much ghosting has built up.

Worked example: with a 65% partial-refresh gain, a pixel converges to within 5% of a fixed target in 3 refreshes — but if that same pixel's target changes again after just 1 refresh (like a real changing clock digit), it never gets the chance to converge at all.

Lesson 3 — A systemd Timer, Not a Busy-Loop

This display only genuinely needs to change a handful of times a day — weather updates every 15-30 minutes, calendar/transit on a similar cadence, a clock region once a minute at most. The service driving it should reflect that, not fight it.

- A **systemd timer** paired with a **oneshot service** (fetch data, render, push to the panel, exit) is the right shape: the timer fires on a schedule, the service does one redraw cycle and stops — no process sitting in memory between updates.
- Compared to a plain cron job, a systemd timer can wait for a dependency (like the network) to actually be ready before firing, and it can catch up on a missed scheduled run after boot, instead of just silently skipping it the way cron does.
- Compared to a busy-loop, a timer-triggered oneshot service uses real power and API quota only when something might actually need to change.

The trap: treating this like a normal display that redraws continuously. E-ink is designed to be written rarely, not polled — a busy-loop or an overly-frequent cron job wastes power and API calls for information that isn't changing that fast in the first place.

Worked example: a naive loop polling a weather API every 5 seconds makes over 17,000 unnecessary requests a day for data that realistically changes every 15-30 minutes; a systemd timer firing every 15 minutes gets the same practical freshness with under 100 requests a day.

Lesson 4 — Designing a Layout for a Screen That Updates Slowly

This is meant to be glanced at, not interacted with — closer to a picture frame than a monitor. That changes what "good layout" even means here.

- Design for **information hierarchy at a glance**: the thing you'd want to see first (time, or the next calendar event) should be the most visually prominent, not buried in a grid of equally-weighted widgets.
- **Deliberately split the layout into regions** matched to how often each one's content actually changes: a large area for weather/calendar/transit (full refresh) and a small dedicated area for a clock (partial refresh, much more often).
- **Panel size trades off against content density**: a 2.13" panel is realistically clock/status only; a 7.5" panel supports a full weather+calendar+transit layout, at the cost of a slower full refresh simply because there's more panel to redraw.

The trap: redrawing the entire panel with a full refresh for every small change, needlessly paying the multi-second full-refresh cost instead of designing a dedicated partial-refresh region for whatever actually changes often.

Worked example: a real layout split — the top 80% of the panel redrawn via full refresh every 15 minutes, and a small clock strip along the bottom redrawn via partial refresh once a minute — gets both a clean, ghost-free main layout and a responsive clock, without ever paying the full-refresh cost once a minute.

Reflection

What's a piece of "always-on" information in your own life that would genuinely benefit from a glanceable, rarely-updating display instead of a phone screen you have to unlock? Why does treating this project's redraw logic as "a service that mostly sleeps" matter more here than on almost any other Chain K project?